Mảng và danh sách liên kết

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 1: Cấu trúc dữ liệu tuyến tính.
- Các bài trong chương: Mảng và danh sách liên kết; Ngăn xếp và hàng đợi.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Chọn cấu trúc phù hợp với thao tác truy cập, tìm kiếm và cập nhật.

Kiến thức nền

Mảng lưu các phần tử ở vùng nhớ liên tiếp trong mô hình cơ bản. Truy cập theo chỉ số có thời gian O(1), nhưng tìm một giá trị trong mảng chưa sắp xếp thường cần O(n). Chèn vào giữa mảng có thể phải dịch chuyển nhiều phần tử nên thường là O(n).

Danh sách liên kết gồm các nút, mỗi nút chứa dữ liệu và liên kết. Danh sách đơn có liên kết tới nút sau; muốn tới phần tử thứ k phải đi từ đầu nên thường O(k). Nếu đã biết nút đứng trước vị trí cần chèn, việc nối một nút mới có thể O(1), chưa tính thời gian tìm vị trí.

Không có cấu trúc luôn tốt nhất: mảng phù hợp truy cập ngẫu nhiên và thường có tính cục bộ bộ nhớ tốt; danh sách liên kết phù hợp một số thao tác nối/tách khi đã có vị trí. Big-O mô tả tốc độ tăng chi phí, không phải số giây chính xác.

1. Mô hình lưu trữ và giới hạn giả định

Trong mô hình mảng cơ bản, phần tử có chỉ số và truy cập trực tiếp. Python list là mảng động các tham chiếu, không hoàn toàn giống mảng số nguyên tĩnh trong C. Vì vậy cần nêu ngôn ngữ và cấu trúc cụ thể khi bàn về bộ nhớ. append của list có thời gian phân bổ bình quân O(1), không có nghĩa mọi lần mở rộng đều hằng số.

Danh sách đơn lưu liên kết next. Biết một nút không đồng nghĩa biết ngay nút trước nó; thao tác xóa cần xét vị trí và mô hình nút. Không khẳng định mọi chèn/xóa danh sách đều O(1) nếu còn phải tìm vị trí.

2. Mô phỏng cập nhật

```
Mảng trước: [A,B,C,D]; chèn X ở chỉ số 2
Dịch D sang phải -> dịch C sang phải -> đặt X
Mảng sau: [A,B,X,C,D]

Danh sách: A -> B -> C -> D
new.next = B.next
B.next = new
Kết quả: A -> B -> X -> C -> D
```

Thứ tự cập nhật liên kết quan trọng: nếu ghi đè B.next mà chưa giữ đường tới C, có thể làm mất phần đuôi khỏi chuỗi truy cập.

3. Ví dụ thực thi Python

```
def insert_copy(values, index, item):
    if not 0 <= index <= len(values):
        raise IndexError("invalid insertion index")
    return values[:index] + [item] + values[index:]

assert insert_copy([1, 3], 1, 2) == [1, 2, 3]
assert insert_copy([], 0, 8) == [8]
```

Hàm tạo bản sao, vì vậy O(n) cả thời gian và bộ nhớ phụ; không nhầm với cập nhật tại chỗ.

4. Bài tập và lời giải

Chọn cấu trúc cho đọc phần tử thứ k rất thường xuyên: mảng là ứng viên hợp lý. Với nối nút khi đã có vị trí, danh sách có lợi thế nhưng vẫn cần quản lý bộ nhớ và liên kết. Nộp bảng truy cập, tìm kiếm, chèn giữa, xóa, kèm giả định; trường hợp rỗng và chỉ một phần tử phải được thử riêng.

> **Bài tập vận dụng bổ sung**
>
> mô phỏng chèn X vào giữa [A,B,C,D], rồi mô phỏng thao tác đó với danh sách liên kết. Đếm thao tác phải thay đổi.

Câu hỏi tự kiểm tra

1. Truy cập phần tử theo chỉ số trong mảng cơ bản có độ phức tạp nào?
2. Tìm một giá trị trong mảng chưa sắp xếp có trường hợp xấu nhất là gì?
3. Danh sách liên kết đơn cơ bản có đặc điểm nào?
4. Chèn vào giữa mảng thường cần thao tác gì?
5. Khi đã biết nút đứng trước, nối một nút mới vào danh sách đơn thường là gì?

Gợi ý đáp án và điểm cần nhớ

- O(1). Địa chỉ phần tử được tính trực tiếp từ chỉ số.
- O(n). Có thể cần kiểm tra tất cả n phần tử.
- Mỗi nút có dữ liệu và liên kết tới nút tiếp theo. Các nút nối với nhau bằng liên kết, không yêu cầu liên tiếp trong bộ nhớ.
- Dịch chuyển các phần tử phía sau. Dịch phần tử tạo chỗ trống nên thường tốn O(n).
- O(1), chưa tính tìm vị trí. Chỉ cần thay đổi một số hữu hạn liên kết.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html